# RISC201 Processor Design and Simulation Framework

**Technical Report**

CSC-201 Computer Organization and Architecture ~ Autumn 2026

**Team Members**: NANDISH MEHRA (25114061)
PREMKUMAR RAHANGDALE (25114072)
MAHENDRA KALA (25114056)
GAURAV LAMBA (25114037)
KANISHK SINGH (25114049)
SHREY PAWAN AGARWAL (25114086)

**GitHub link**: https://github.com/Premkumar-Rahangdale/Processor_Design.git

## Executive Summary

RISC201 is a software-simulated, 32-bit processor with fixed-width instructions, a 32-entry register file, word-indexed data memory, and 25 defined opcodes. The project implements four-stage and six-stage in-order pipelines, hardwired and microprogrammed control unit, an integer and floating arithmetic logic unit, stack support, and an interactive terminal environment called P.E.K.K.A.

The four-stage pipeline combines execution and memory access. The six-stage pipeline separates register read and memory access, which reduces work within those stages but adds pipeline boundaries. Both designs share the same architectural state and ALU. Horizontal and vertical microprogrammed control unit use the same instruction routines in different control-word formats.

The software toolchain prepares source code, expands stack macros, assembles 32-bit words, disassembles machine code, and supports pipeline-level and microinstruction-level debugging. In P.E.K.K.A., `debug` steps the configured pipeline. When a microprogrammed controller is selected, `execute` runs the stand-alone microengine on its own datapath and reports microinstruction cycles; pipeline execution with microprogram-derived control signals is available through the pipeline interface used by the debugger.

## Project Overview

The CSC-201 assignment combines processor organization with a working simulation environment. This project represents a processor as C++ state and stage logic so that instruction formats, control signals, pipeline behavior, micro-operations, stack checks, and debugging output can be inspected directly.

| Design area | Project implementation |
|---|---|
| Instruction set | RISC201: 25 opcodes, 32-bit instruction words, register and signed-immediate operands, relative control-transfer offsets. |
| Processor pipelines | Cycle-accurate in-order 4-stage and 6-stage organizations using a shared pipeline engine. |
| Control | Hardwired decoding, plus horizontal and vertical microprogrammed control units with a common microprogram. |
| Execution units | 32-bit integer ALU, IEEE-754 floating arithmetic for selected register operations, separate instruction and data memories. |
| Software environment | Loader and stack preprocessor, two-pass assembler, disassembler, P.E.K.K.A. CLI, pipeline debugger, microdebugger, and stack visualizer. |

| Design area | Project implementation |
|---|---|
| Evaluation | Run-time counters for cycles, CPI, stalls, branches, prediction, squashes, control bubbles, and microinstruction activity. |

The machine models architectural behavior and cycle counts. Conclusions about pipeline trade-offs are therefore expressed in stage structure and simulated cycles rather than measured hardware clock frequency.

## RISC201 Instruction Set Architecture

RISC201 uses 32-bit words and a five-bit opcode. The instruction word contains fields for an immediate selector, destination and source registers, and either a second register, a signed immediate, or a relative offset. The ISA has 25 assigned opcode values, leaving seven encodings in the five-bit opcode space unused.

## Instruction Formats

| Format | Fields | Purpose |
|---|---|---|
| Register | 31-27 opcode \| 26 I=0 \| 25-21 rd \| 20-16 rs1 \| 15-11 rs2 \| 10-0 reserved | Three-register arithmetic, logic, shift, and data-transfer forms. |
| Immediate | 31-27 opcode \| 26 I=1 \| 25-21 rd \| 20-16 rs1 \| 15-0 signed immediate | Register plus signed 16-bit immediate; also used for memory offsets. |
| Two-operand | Opcode and I bit, with unused register fields set to zero and operand B in the low field | Used by cmp and the mov/not family; the exact active fields depend on the mnemonic. |
| Control transfer | 31-27 opcode \| 26-0 signed relative offset | beq, bgt, bsm, b, and call target an instruction index relative to the current instruction. |

For register-form arithmetic, the 11 low-order bits after rs2 are zero. For immediate form, the low 16 bits hold a signed value. A load or store adds its signed 16-bit displacement to the base register to produce a word address. Branch and call offsets are 27-bit signed values; the simulator uses instruction-vector indices rather than byte addresses.

## Opcode Map

| Opcode | Mnemonic | Opcode | Mnemonic |
|---|---|---|---|
| 00001 | add | 01110 | lsl |
| 00010 | sub | 01111 | lsr |
| 00011 | mul | 10000 | asr |
| 00100 | div | 10001 | nop |
| 00101 | mod | 10010 | ld |
| 00110 | cmp | 10011 | st |
| 00111 | and | 10100 | beq |
| 01000 | or | 10101 | bgt |
| 01001 | not | 10110 | bsm |
| 01010 | xor | 10111 | b |

| Opcode | Mnemonic | Opcode | Mnemonic |
|---|---|---|---|
| 01011 | mov | 11000 | call |
| 01100 | movu | 11001 | ret |
| 01101 | movh | | |

The operation groups are arithmetic and comparison (add, sub, mul, div, mod, cmp), bitwise logic (and, or, xor, not), data transfer and immediate construction (mov, movu, movh), shifts (lsl, lsr, asr), memory (ld, st), control transfer (beq, bgt, bsm, b, call, ret), and nop.

## Operand and Condition Behavior

- add, sub, mul, div, mod, and, or, xor, and the shifts write rd. Their third operand can be a register or a signed 16-bit immediate where the encoding permits it.
- cmp subtracts the second operand from the first and updates the condition flags; it does not write a general destination register. beq tests equality, bgt tests signed greater-than, and bsm tests signed less-than.
- mov copies its B operand. movu keeps the low 16 bits and clears the upper half. movh places the low 16 bits of B in bits 31-16 and clears bits 15-0. not complements B. Shift amounts come from the low five bits of B.
- ld rd, displacement[base] loads one data word. st rd, displacement[base] stores the value of rd. The address is base plus the signed displacement.
- call transfers to a relative target and records the following instruction index in r1. ret transfers to the address in r1. The software stack preprocessor preserves the link around nested calls.

The register file also carries floating-point values as raw 32-bit patterns. When add, sub, mul, or div is in register form and both source registers are in r26-r31, the decoder selects floating arithmetic. Immediate forms, mixed register ranges, comparisons, modulo, and memory operations use integer behavior.

## Processor Organization

The simulator's MachineState is the architectural state shared by the pipelines and the microdatapath. It contains 32 integer register slots, four condition flags, a data-memory vector, an instruction-memory vector, label maps, and the instruction program counter. Register and data-memory values are 32-bit integers in the implementation; instruction words are held as 32-character binary strings, with label entries stored separately in the same instruction vector.

## Register Organization

| Register | Code | Convention | Implementation behavior |
|---|---|---|---|
| r0 | 00000 | Constant zero | Reads as 0; writes are ignored. |
| r1 / ra | 00001 | Return address | call writes the link value; ret reads it. |
| r2 / sp | 00010 | Stack pointer | Full-descending stack; empty value is 4096. |
| r3 / fp | 00011 | Frame pointer convention | FP marks the base of a function’s stack frame, making its local variables and saved registers easier to access. |
| r4 | 00100 | Exception cause | Receives the exception code when a precise pipeline fault commits. |
| r5 | 00101 | Faulting instruction index | Receives the faulting instruction index on pipeline or microengine exception handling. |
| r6-r25 | 00110-11001 | General-purpose | Twenty unreserved integer registers. |

| Register | Code | Convention | Implementation behavior |
|---|---|---|---|
| r26-r31 | 11010-11111 | Floating-point operand range | The same register slots hold raw floating values for selected add, sub, mul, and div operations; they remain 32-bit register values for other instructions. |

## Memory and Program Counter

| Resource | Size or range | Use |
|---|---|---|
| Instruction memory | Program-length vector | Stores encoded words and label entries. Fetch skips label entries without consuming a clock. |
| Data memory | 4096 word indices, 0-4095 | Separate from instruction memory; loads and stores use word indices. |
| Reserved stack | 3072-4095 | The full-descending stack occupies the top 1024 data words. |
| Empty stack pointer | 4096 | One past the highest stack address; the first push decrements SP to 4095. |

The program counter indexes the instruction vector. The .main label is the preferred entry point; if it is absent, execution begins at index 0. Control-transfer offsets are relative to the current instruction index. The stack occupies data memory, while instruction fetch reads from the independent instruction vector.

## Datapath Sequence

At a high level, the processor fetches an instruction, decodes its opcode and operand fields, reads the required registers, performs an ALU or address operation, optionally accesses data memory, and writes a result in WB. The stage boundaries differ between the two pipelines, but they share the same architectural register and memory state.

## Four-Stage and Six-Stage Pipelines

A single in-order pipeline engine is parameterized with two stage layouts. Each clock advances different instructions through the stages. Instructions commit in program order, which limits the principal data dependency to read-after-write hazards.

| Stage | Four-stage pipeline | Six-stage pipeline |
|---|---|---|
| IF | Fetch 32-bit word; advance or redirect PC. | Fetch 32-bit word; advance or redirect PC. |
| ID | Decode control and read operands. | Decode opcode and instruction fields. |
| RR | Included in ID. | Read source registers and prepare immediate operand. |
| EX / EX-MEM | ALU, branch decision, address calculation, and memory access share EX/MEM. | ALU, branch decision, and effective-address calculation occur in EX. |
| MEM | Included in EX/MEM. | Loads read and stores write data memory. |
| WB | Write a register result or commit an exception. | Write a register result or commit an exception. |

The stage flows are `IF -> ID -> EX/MEM -> WB` and `IF -> ID -> RR -> EX -> MEM -> WB`. Four-stage combines more work in each execution stage and has fewer internal boundaries. Six-stage separates register read and data

memory from neighboring work, adding pipeline registers and more opportunities for hazards. Either design can ideally complete one instruction per clock after fill, subject to hazards and redirects. An ideal N-instruction run with S stages takes N + S - 1 clocks before stalls or control penalties.

## Hazards and Branch Control

The engine detects RAW dependencies on source registers, store data, and the return-address register. It can resolve dependencies by stalling or by forwarding results from older instructions. Forwarding is enabled in the P.E.K.K.A. CLI; the underlying pipeline can also be configured for stall-only or write-through behavior. r0 is excluded from hazard checks because it always reads as zero.

Branches, calls, and returns resolve in the execution stage: EX/MEM in the four-stage layout and EX in the six-stage layout. The simulator supports predict-not-taken and a two-bit saturating predictor with a branch target buffer. P.E.K.K.A. fixes the two-bit predictor as its default. A misprediction squashes younger instructions and redirects fetch; the documented redirect penalty is two clocks for four stages and three clocks for six stages.

Only cmp changes the condition flags. It computes subtraction information and produces zero, negative, overflow, and carry flags. Signed greater-than and less-than are derived as `!Z && N == O` and `N != O`, allowing bgt and bsm to test the result. A branch following cmp can use the condition produced by that comparison.

## Precise Exceptions

Pipeline faults travel with their instruction until WB. Older instructions finish first, the faulting instruction is prevented from making a normal architectural update, and younger instructions are squashed. The simulator records the cause in r4, the instruction index in r5, prints the fault, and stops; it does not dispatch to a resumable exception handler.

| Code | Cause | Detection or rule |
|---|---|---|
| 1 | Illegal opcode or malformed word | Detected during fetch or decode. |
| 2 | Integer division or modulo by zero | Detected by the ALU in EX. |
| 3 | Data address outside memory | Detected on data-memory access. |
| 4 | Stack overflow | SP is written below 3072, or a store through r2 leaves the stack region. |
| 5 | Stack underflow | A load through r2 addresses 4096 while the stack is empty. |
| 6 | Stack-pointer corruption | A write to r2 sets SP above 4096. |
| 7 | Stack-address violation | A load through r2 leaves 3072-4095. |

## Microprogrammed Control Unit

The control unit translates each instruction into actions such as register transfers, ALU selection, memory access, branch selection, and exception handling. The project provides a hardwired decoder and two microprogrammed encodings. Both microprogrammed units load the same RISC201 microprogram, which is assembled into control memory and sequenced by a microprogram counter (microPC).

## Microdatapath and Control Memory

The stand-alone microdatapath contains 24 micro-registers, including the instruction fields, sign-extended immediate, branch target, register-file address and data, ALU operands and result, flags, memory address and

data, load result, and exception cause. One shared bus transfers values. An `out` control selects a bus source, a transfer multiplexer chooses the bus, bus-plus-immediate, or immediate, and `in` controls load one or more destinations. An `args` field selects the functional unit operation for that microinstruction.

| Microinstruction | Action |
|---|---|
| mloadIR | Fetch instruction memory into the micro instruction register. |
| mdecode | Decode opcode, register fields, immediate, and relative target. |
| mswitch | Dispatch the microPC to the routine for the instruction opcode. |
| mmov dst, src | Transfer a micro-register value to another micro-register. |
| mmovi dst, imm | Load a micro-immediate into a micro-register. |
| madd dst, imm | Add a micro-immediate to the selected micro-register. |
| mbeq src, imm, label | Branch within microcode when a micro-register equals an immediate. |
| mb label | Unconditionally set the microPC to a microcode label. |

The common instruction preamble loads IR, decodes the fields, increments the machine PC, and dispatches through the opcode switch. Arithmetic routines read operands, invoke the ALU, and write back the destination. Load/store routines calculate an address before using the memory unit. Branch routines test condition flags and update the machine PC. Call writes the following instruction index to r1; ret reads r1 and redirects PC.

## Horizontal and Vertical Formats

| Property | Horizontal control word | Vertical control word |
|---|---|---|
| Word width | 70 bits | 45 bits |
| Fields | 38 direct control bits + 12-bit immediate + 10-bit target + 10-bit args | 3-bit type + 5-bit source + 5-bit destination + 12-bit immediate + 10-bit target + 10-bit args |
| Decode | Control signals are stored directly in the word. | A microdecode stage expands the compact type and register fields. |
| Trade-off | Wider words, direct control, no separate microdecode stage. | Narrower words, additional decode stage. |
| Control-store use | 196 of 1024 words; 13,720 stored bits. | 196 of 1024 words; 8,820 stored bits. |

The microPC addresses a 1024-word control memory. The current microprogram occupies 196 words. The horizontal store uses about 56 percent more bits for those words than the vertical store, while the vertical unit adds one microdecode stage. The stand-alone microsequencer records microinstruction count, cycles, redirects, faults, and microPC state.

## Pipeline Integration and Microdebugging

The pipeline's controlunit interface accepts the six-bit opcode-plus-I field and returns the control signals needed by ID. With hardwired control, Boolean decode logic generates those signals. With `useMicrocode()`, the selected control memory is inspected to derive the instruction signals. Horizontal and vertical control can therefore be attached to the pipeline decoder, including in the pipeline debugger.

The stand-alone `run()` path executes one microinstruction per clock on the microdatapath, without a pipelined instruction engine. P.E.K.K.A. exposes this route through `microdebug` and `microcode`; when a microprogrammed controller is selected, the CLI's `execute` command uses this stand-alone engine. This distinction matters when comparing pipeline cycles with microinstruction cycles.

The microdebugger can step one microinstruction with `ustep`, advance by a complete guest instruction with `step`, run to the end, restart, inspect `state` and `uregs`, view architectural registers/memory/stack, and list the encoded control store. Each step reports the current microPC, operation, next-address decision, clock count, and changed micro-register or architectural state.

## RISC201 Arithmetic Logic Unit

The ALU accepts two 32-bit operands and the decoded control signals, then returns a result, status flags, a flag-update indication, and a division-by-zero indication. It is shared by both pipeline organizations and by the microdatapath. The integer datapath includes arithmetic, comparison, logical operations, shifts, and moves; an IEEE-754 floating unit handles selected floating-point register operations.

| Operation | Instruction | Implementation and behavior |
|---|---|---|
| Addition | add; ld/st address | 32-bit Han-Carlson parallel-prefix adder; effective address is base + displacement. |
| Subtraction | sub; cmp | A + not(B) + 1. cmp also derives Z, N, O, and C status. |
| Multiplication | mul | Radix-4 Booth recoding and Wallace reduction; the architectural result is the low 32 bits. |
| Division and remainder | div; mod | One signed non-restoring divider produces quotient or remainder; integer division truncates toward zero. |
| Logic | and; or; xor; not | Bitwise operations on A and B; not complements B. |
| Shifts | lsl; lsr; asr | Five-layer barrel shifter; shift amount is B[4:0]. Logical shifts fill with zero; arithmetic right shift extends the sign bit. |
| Moves | mov; movu; movh | Forward B; retain low 16 bits; or place low 16 bits in the upper half, respectively. |

## Arithmetic Structures

For addition and subtraction, the Han-Carlson adder forms per-bit generate and propagate terms and combines them in parallel-prefix levels. The project documentation describes six levels and 80 prefix cells for a 32-bit network, compared with a 129-cell Kogge-Stone layout. This is an algorithmic implementation detail in the simulator, not a measured physical circuit result. Subtraction reuses the adder by complementing B and setting carry-in to one.

The integer multiplier uses radix-4 Booth recoding to form fewer partial-product rows, followed by a Wallace tree and a final 32-bit adder. Only the low 32 product bits are constructed for the architectural result. The signed non-restoring divider maintains a widened partial remainder, calculates quotient and remainder together, then restores signs; the remainder follows the sign of A. Division by zero raises the integer runtime fault. The `INT_MIN / -1` quotient wraps to `INT_MIN` with remainder zero; its overflow indication is not committed as an architectural flag.

## Flags and Floating-Point Operations

The integer adder produces zero, negative, overflow, and carry information. The machine flags are updated only by cmp; add and sub do not latch their overflow results. beq uses equality, bgt uses signed greater-than, and bsm uses signed less-than. `divByZero` signals an exception rather than a normal ALU result.

The floating unit is selected for register-form add, sub, mul, and div when both sources lie in r26-r31. It supports subnormal values and round-to-nearest, ties-to-even. It does not update integer flags and does not raise the integer division-by-zero fault; division by floating-point zero returns signed infinity. The simulator models every ALU operation as one EX-stage clock, including integer division and the floating-point slow paths.

## Stack and Procedure Support

The stack is full-descending: an empty stack has SP = 4096, a push decrements SP and stores at the new address, and a pop loads at SP before incrementing it. Valid stack words are 3072 through 4095. The stack is held in ordinary data memory and uses r2 as its pointer.

| Source construct | Expanded instruction sequence |
|---|---|
| push rN | sub r2 r2 1; st rN 0[r2] |
| pop rN | ld rN 0[r2]; add r2 r2 1 |
| call .function | Save registers identified as used by the caller, issue call, then restore those values. |
| Function entry and ret | Save r1 at entry; reload it and advance SP immediately before ret. |

The loader invokes a source preprocessor that scans .main and labels referenced by call. It identifies registers used within function regions and emits stack stores around calls; it also saves the return address at function entry. Since the ISA has no halt instruction, the preprocessor inserts a branch around out-of-line function bodies when main would otherwise fall through into them. push and pop accept a single valid register and reject r0 and r2.

Stack checks are implemented in the simulator's exception paths. Writes that move SP below 3072 signal overflow, values above 4096 signal pointer corruption, and loads/stores using r2 are checked against the reserved stack range. A load through r2 at 4096 is underflow. General data accesses are separately checked against the 0-4095 memory bounds. The ASCII stack visualizer reports the current SP, bounds, visible values, empty state, and top-of-stack marker; it is available in both debuggers.

## Loader and Stack Preprocessor

P.E.K.K.A. loads the assembly source from `tests/assembly_program.txt`. The loader removes `//` comments, collapses repeated whitespace, skips blank lines, strips a UTF-8 byte-order mark at the start of the file, and passes the remaining lines through the stack preprocessor. The expanded source is written to `tests/t_asmblr.txt` for the assembler.

Labels use a dot-prefixed name on a line of their own, such as `.main:` or `.loop:`. Operands are space-separated; common examples are `add r5 r5 r6`, `mov r6 10`, `ld r7 0[r2]`, `call .function`, and `ret`. The assembler recognizes `ra` as r1 and `sp` as r2.

## Two-Pass Assembler

The first pass records label locations. The second pass encodes each operation into a 32-character binary word, handles register and immediate forms, pads unused fields, and resolves branch/call labels to signed relative offsets. Machine words are added to the simulator's instruction memory and written to `tests/o_asmblr.txt`. The current assembler emits binary output.

For example, the immediate instruction `add r6 r7 12` uses opcode 00001, I=1, rd=00110, rs1=00111, and a signed 16-bit immediate whose binary value is 0000000000001100. In register form, the low operand field carries rs2 instead of the immediate.

## Disassembler

The disassembler reads the binary word stream, extracts the opcode, I bit, register fields, signed immediate, or signed branch offset, and formats the instruction as RISC201 assembly. Where the assembler's reverse label map is available, it prints symbolic branch targets. The result is written to `tests/o_disasmblr.txt`. Together, the assembler and disassembler provide a source-to-word-to-source inspection path for the implemented encodings.

## PEKKA Command Line and Debuggers

P.E.K.K.A. is the interactive terminal front end that connects the loader, assembler, disassembler, machine state, pipelines, control units, and debuggers. At startup the user selects four or six pipeline stages and hardwired, horizontal, or vertical control. The CLI uses forwarding and the two-bit predictor with BTB as fixed settings; the lower-level pipeline interface exposes other hazard and predictor modes.

| Command | Purpose |
|---|---|
| start | Power on and select stage count and control unit. |
| config | Show pipeline and control configuration. |
| load | Clean assembly input and expand stack macros. |
| assemble | Encode the loaded source into binary machine words. |
| disassemble | Reconstruct assembly text from assembled words. |
| execute | Run the hardwired pipeline, or run the stand-alone microengine when microprogrammed control is selected. |
| debug | Step the selected pipeline and inspect architectural changes. |
| microdebug | Step the configured horizontal or vertical microengine. |
| microcode | List the selected control memory and microprogram. |
| help, clear, exit | Show help, clear the terminal, or leave the CLI. |

## Pipeline Debugger

The pipeline debugger advances one or more whole instructions, can restart and show the first N instructions, and provides register, memory, stack, and state views. After a step it reports which instructions executed, differences in registers and data memory, changed condition flags, the program counter, and the next instruction. This exposes architectural effects while the pipeline engine handles stage timing, forwarding, prediction, and exceptions.

## Microinstruction Debugger

The microdebugger operates on the stand-alone microdatapath. `ustep [n]` advances microinstructions; `step [n]` advances guest instructions; `run` continues to completion; `restart` resets state. `state`, `uregs`, `registers`, `memory`, `stack`, `listing`, and `words` expose the microPC, micro-registers, architectural state, and encoded control words. A micro fault transfers control to the microcode exception routine, which writes r4 and r5 before halting.

## Evaluation Framework and Design Tradeoffs

The project exposes counters that support comparisons across pipeline length, control unit, assembly programs, and branch behavior. P.E.K.K.A. prints the instruction count, clock count, CPI, data-stall cycles, branches, mispredictions, prediction accuracy, squashed instructions, and control-path bubbles for pipeline runs. The microengine reports instruction count, microinstruction count, cycles, CPI, word width, and control-store usage.

| Measure | What it describes | Comparison use |
|---|---|---|
| Cycles and CPI | Total clock cycles and cycles per completed instruction. | Compare the same program under four and six stages or across control modes. |
| Data stalls | Clocks inserted to resolve register dependencies. | Show the effect of forwarding, load-use dependencies, and stage placement. |

| Measure | What it describes | Comparison use |
|---|---|---|
| Branches and mispredictions | Control transfers resolved and predictions found wrong. | Measure prediction behavior on loops and procedure calls. |
| Squashed instructions | Younger wrong-path instructions discarded after redirect. | Describe the cost of control hazards. |
| Control bubbles | Cycles lost while a selected control path refills. | Expose the vertical microdecode penalty and pipeline redirects. |
| Microinstruction count | Micro-operations executed by the stand-alone sequencer. | Compare routine length and control-memory behavior. |
| Control-store bits | Used words multiplied by horizontal or vertical width. | Compare direct control width with compact encoding. |

Design-level comparison favors the four-stage organization when simpler timing and lower redirect latency matter. The six-stage organization separates work more finely, but its longer pipeline increases the number of stages that can contain younger instructions and raises the documented redirect cost. Which design is faster depends on the instruction mix and hazard behavior; this simulator does not calculate a hardware clock period.

Horizontal microcode uses wider control words but has direct control fields and no microdecode stage. Vertical microcode stores fewer control bits and adds a decode stage. Both use the same 196-word microprogram and represent the same instruction behavior. A fair run-time comparison should keep the assembly program and initial state fixed, record the selected pipeline and control mode, and interpret pipeline cycles separately from stand-alone microengine cycles.

CPI is workload-dependent rather than a fixed processor constant. The CLI and pipeline interface provide the counters needed to report measured results for selected programs; this report describes those measures and the implementation's structural trade-offs without assigning a benchmark score to a run that is not present in the project materials.

## Conclusion

RISC201 combines an inspectable instruction set with two pipeline organizations, a common ALU, switchable hardwired or microprogrammed control, and a software environment that makes both architectural and microarchitectural state visible. The four-stage and six-stage variants expose the effects of combining or separating pipeline work. Horizontal and vertical microcode show the storage-versus-decode trade-off, while P.E.K.K.A. connects the architecture to assembly preparation, execution, statistics, and step-by-step debugging.

The main architectural boundary is clear: the project is a simulator for comparing organization and control behavior, not a timing-accurate hardware model. Its counters make workload-dependent pipeline and microprogram comparisons possible, and its debugger views help explain how each instruction changes machine state.